Docker Practical Learning Guide

Docker Practical Learning Guide — Barnx Studio Edition

From beginner fundamentals to a Dockerized Node.js + PostgreSQL project

Built around the practical project: **Docker Student API** — Node.js, Express, PostgreSQL, Docker, Docker Compose, volumes, networks, health checks, environment variables and Git/GitHub.

Repository: https://github.com/Mrbarnx/docker-student-api

How to use this guide

Use the same learning pattern throughout:

Concept → Simple Explanation → Command/Code → Breakdown → Plain-English Meaning → Why It Matters → Practical Task → Expected Result → Checkpoint

Do not try to memorize every command. Run the commands yourself, understand what changed, and debug the exact error before changing unrelated things.

Week 1 — Docker Foundations

1. Docker in plain English

Docker packages an application together with the environment it needs and runs it inside an isolated container.

Core mental model

```
Dockerfile
   ↓ docker build
Image
   ↓ docker run
Container
   ↓
Running application
```

- **Image:** reusable package/blueprint.
- **Container:** running or stopped instance created from an image.
- **Dockerfile:** instructions Docker follows to build an image.
- **Docker Compose:** defines and manages multiple services in `compose.yaml`.
- **Volume:** persistent Docker-managed storage.
- **Network:** communication layer between containers.

2. Prove Docker works

```
docker version
docker run hello-world
```

Plain English: Ask Docker for its client/server version, then download and run the small `hello-world` test image.

Checkpoint: You should see Docker's hello-world success message.

3. Images and containers

```
docker ps
docker ps -a
docker images
```

- `docker ps` → running containers only.
- `docker ps -a` → all containers, including stopped ones.
- `docker images` → images stored locally.

4. Run Nginx and learn ports

```
docker run -d --name my-nginx -p 8081:80 nginx
```

Breakdown:

- `docker run` → create a new container.
- `-d` → run in the background.
- `--name my-nginx` → give the container a readable name.
- `-p 8081:80` → forward laptop port `8081` to container port `80`.
- `nginx` → use the Nginx image.

Plain English: Create an Nginx container, run it in the background, call it `my-nginx`, and make it available at `localhost:8081`.

Practice: Open `http://localhost:8081` in your browser.

5. Logs, inspection and entering a container

```
docker logs my-nginx
docker inspect my-nginx
docker exec -it my-nginx sh
```

Use logs first when something fails. `docker exec` lets you run a command inside an already-running container.

Week 1 checkpoint

You should be able to explain:

- image vs container
- `docker run` vs `docker start`
- host port vs container port
- how to inspect logs
- how to stop and remove a container safely

Week 2 — Build Your Own Container

1. Start with a small Node.js app

A Dockerfile describes how to package the app.

```
FROM node:22-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Line-by-line meaning

- `FROM node:22-slim` → start from a small image that already contains Node.js.
- `WORKDIR /app` → use `/app` as the working directory.
- `COPY package*.json ./` → copy dependency files first.
- `RUN npm ci` → install exact locked dependencies.
- `COPY . .` → copy the application source.
- `EXPOSE 3000` → document the app's container port.
- `CMD ...` → start the application when the container starts.

2. Add `.dockerignore`

```
node_modules
npm-debug.log
.git
.gitignore
.env
```

`.dockerignore` controls what Docker sees during a build. `.gitignore` controls what Git tracks.

3. Build and run your image

```
docker build -t day2-node-app .
docker run -d --name day2-node-container -p 3000:3000 day2-node-app
```

Plain English: Build an image from the Dockerfile in the current folder, then create a container from that image and expose the app on port 3000.

4. Docker Compose

Compose moves long runtime configuration into a reusable YAML file.

```
services:
  app:
    build: .
    ports:
      - "3000:3000"
```

Useful commands:

```
docker compose up -d
docker compose up -d --build
docker compose ps
docker compose logs app
docker compose down
```

Warning: `docker compose down -v` also removes named volumes and may delete database data.

5. Environment variables

Keep configuration outside the application source.

```
PORT=3000
HOST_PORT=5000
```

```
        ports:
          - "${HOST_PORT}:${PORT}"
```

Checkpoint: Change the host port through `.env`, recreate the service, and verify the app is available on the new port.

Week 3 — Docker Compose + PostgreSQL + Final Project

1. Container networking

Inside a container, `localhost` means that same container. In Docker Compose, services can reach each other using their service names.

For the Student API:

```
DB_HOST=db
DB_PORT=5432
```

`db` is the PostgreSQL service name.

2. Persistent PostgreSQL data

```
        services:
          db:
            volumes:
              - postgres_data:/var/lib/postgresql/data

        volumes:
          postgres_data:
```

Plain English: Store PostgreSQL data in a Docker-managed named volume so the data can survive container recreation.

3. Clone the real project

```
git clone https://github.com/Mrbarnx/docker-student-api.git
cd docker-student-api
```

Create a local `.env` from `.env.example`, then run:

```
docker compose up -d --build
docker compose ps
```

Expected services:

```
student-api
student-db
```

The PostgreSQL service should become healthy.

4. Final architecture

```
Client / Browser
      ↓
localhost:5001
      ↓
Node.js + Express container
      ↓
Docker network
      ↓
PostgreSQL container
      ↓
Named PostgreSQL volume
```

Test:

- `GET /` → confirms the API is running.
- `GET /health` → health response.
- `GET /students` → reads students.
- `POST /students` → creates a student using a parameterized SQL insert.

5. Prove persistence

```
docker compose down
docker compose up -d
```

If previously stored students still exist, the named volume is working.

Do **not** use `docker compose down -v` unless you intentionally want to delete the learning database volume.

Debugging — Do Not Guess

When something fails:

1. Read the exact error.
2. Run `docker compose ps` or `docker ps`.
3. Read logs with `docker compose logs <service>` or `docker logs <container>`.
4. Validate Compose with `docker compose config`.
5. Check environment values, service names and ports.
6. Check networks/volumes when the problem involves connectivity or persistence.
7. Change only the thing supported by the evidence, then retest.

Common examples:

- **Port already allocated:** another process/container owns the host port.
- **Cannot connect to Docker daemon:** Docker CLI exists but the Engine is not running.
- **`relation "students" does not exist`:** database connection works, but the table has not been initialized.
- **Source change not appearing:** rebuild with `docker compose up -d --build` or use a bind mount during development.

Skills You Can Honestly List After Completing the Project

Provided you can explain and demonstrate them:

- Docker
- Docker Compose
- Dockerfiles
- Docker images and containers
- Containerization
- Port mapping
- Docker networking
- Named volumes and database persistence
- Environment configuration
- Docker logs / inspect / exec / Compose debugging
- Node.js
- Express.js
- REST API fundamentals
- PostgreSQL
- Basic SQL and parameterized queries
- Git and GitHub

Fair CV wording:

> Docker & Docker Compose — containerized Node.js applications, multi-container development, Docker networking, persistent volumes, environment configuration and health checks.

Docker knowledge alone does not make someone a DevOps Engineer. A fair description is **DevOps fundamentals** or **containerization with Docker/Docker Compose** while continuing into Linux, CI/CD, cloud, infrastructure and monitoring.

Interview Practice

Be able to answer these in your own words:

1. What is the difference between an image and a container?
2. What does `docker run` do?
3. `docker run` vs `docker start`?
4. What is a Dockerfile?
5. Why use `.dockerignore`?
6. What is Docker Compose?
7. Why use a named volume for PostgreSQL?
8. Why does the Node container connect to `db` instead of `localhost`?
9. What is a health check?
10. How would you debug an API container that fails to start?

Practical challenges:

- Containerize a small Node.js app from a blank folder.
- Explain every line of a Dockerfile.
- Write a Compose file for Node.js + PostgreSQL.
- Add a named PostgreSQL volume.
- Diagnose a host-port conflict.
- Inspect logs and enter a running container.
- Demonstrate database persistence after container recreation.

What to Learn Next

After Docker feels comfortable:

1. Linux fundamentals
2. Deploy a Docker project to a VPS
3. Domains + HTTPS + reverse proxy
4. GitHub Actions / CI/CD
5. Cloud fundamentals
6. Monitoring and logging

7. Infrastructure as Code later
8. Kubernetes later, after Docker is comfortable

Final rule: The strongest evidence of Docker skill is not memorizing commands. It is being able to build, debug and explain a containerized project without blindly following a tutorial.